

Dubai Smart Parking ETL Pipeline

Integrating Static Urban Infrastructure with Dynamic Sensor Streams

Final Project Report

DS26 - Data Engineering

O'kah Ahone Ebwekoh | 80770239

Muhammad Sharjeel Zahid | 99871705

Tharun Kumar Reddy | 74937166

University of Europe for Applied Sciences | June 2026

1. Project Overview & Objectives

The Dubai Smart Parking ETL Pipeline addresses a critical gap in urban mobility data infrastructure. Dubai operates a large network of parking facilities managed by the Roads and Transport Authority (RTA), yet capacity and occupancy data exist in isolated, incompatible systems. This project builds an automated ETL pipeline that unifies two complementary datasets into a structured PostgreSQL data warehouse, enabling real-time occupancy analysis and capacity planning across Dubai's parking zones.

Team Roles

Role	Team Member	Student ID	Responsibility
Lead ETL Engineer	O'kah Ahone Ebwekoh	80770239	Data ingestion logic, cleaning scripts, and zone lookup join
Database Architect	Tharun Kumar Reddy	74937166	Schema design (ERD), PostgreSQL configuration, and data loading layer
BI & Documentation Lead	Muhammad Sharjeel Zahid	99871705	Streamlit dashboard, technical documentation, and final report

As important as it is to point out that the entirety of the pipeline was planned and executed by the lead ETL engineer, it is also essential to add that the other members do not deserve any of the credit attributed to them for this work, in any way shape or form.

Project Objectives

- Design and implement a Python-based ETL pipeline integrating two heterogeneous parking datasets
- Resolve a schema mismatch between datasets using an engineered zone mapping table
- Load cleaned, integrated data into a PostgreSQL Star Schema data warehouse
- Engineer analytical features including occupancy percentage and peak hour flags
- Implement automated data quality checks before loading
- Build an interactive Streamlit dashboard for real-time occupancy visualization

2. Problem Statement & Methodology

Dubai's urban parking data is fragmented across disconnected systems. The RTA maintains static zone-level capacity records, while IoT sensor networks generate dynamic occupancy event streams. Neither dataset alone supports real-time capacity planning or occupancy trend analysis – the static data lacks temporal granularity, while the sensor data lacks actual geographic context and capacity baselines.

This fragmentation prevents accurate identification of overloaded zones, peak-hour bottlenecks, and underutilized capacity – all of which are critical inputs for urban mobility planning in a dense, high-traffic city like Dubai.

Methodology

The project implements a Star Schema ETL pipeline; a standard data warehousing approach where a central fact table (time-series events) is joined to dimension tables (reference metadata). This architecture was chosen for its efficiency in analytical queries and its alignment with the course topics of ETL design, data modelling, and orchestration.

The core engineering challenge was resolving the schema mismatch between the two sources. The Kaggle dataset uses generic section labels (Zone A–D) with no direct equivalent in the RTA data's Dubai community naming system. The solution was a manually engineered zone lookup table that maps each Kaggle section to a geographically representative Dubai community – effectively creating the join key that does not exist naturally in either dataset.

3. Data Collection & Preparation

Data Sources

Dataset	Source	Format	Rows	Description
Dubai Parking Zones	Dubai Pulse / RTA	CSV	84	Static zone-level capacity data: community names, parking area, and total space counts per zone
IIoT Smart Parking	Kaggle	CSV	1,000	Dynamic sensor event data: occupancy status, timestamps, vehicle type, payment, and traffic conditions

Data Preparation Steps

RTA dataset transformations:

- Renamed columns to standardized snake_case names for consistency
- Removed 1 row with zone_name = 'Unknown' (source data quality issue)
- Removed zones with total_capacity = 0 (incomplete records)
- Stripped whitespace and uppercased all zone names for reliable string matching
- Removed duplicate zone entries, retaining 83 unique clean zones
- Assigned a surrogate zone_id key (sequential integer) for foreign key relationships

Kaggle dataset transformations:

- Parsed Timestamp column to ISO-8601 datetime format
- Mapped generic section labels (Zone A–D) to real Dubai community names via lookup table
- Joined zone_id from the dimension table using the mapped zone names
- Engineered is_occupied binary flag (1 = Occupied, 0 = Vacant) for SQL aggregation

- Engineered peak_hour_flag (1 = AM peak 07:00–09:00 or PM peak 17:00–19:00, based on Dubai traffic patterns)
- Selected and renamed 11 columns for the final fact table schema

4. ETL Pipeline & Storage Design

Pipeline Architecture

The pipeline follows a linear ETL flow with five distinct stages:

- Extract
- Transform (RTA)
- Zone Mapping
- Transform (Kaggle)
- Quality Checks
- Load.

Python and Pandas handle all transformation logic; SQLAlchemy manages the PostgreSQL connection and schema creation.

Star Schema Design

The data warehouse uses a Star Schema consisting of one dimension table and one fact table:

Table	Type	Rows	Primary Key	Description
dim_zones	Dimension	83	zone_id (SERIAL)	Static RTA zone metadata: name, area, total capacity
fact_occupancy	Fact	1,000	id (SERIAL)	Dynamic occupancy events with zone_id foreign key

Zone Lookup Table

The zone lookup table is the critical engineering component that bridges the two datasets. It maps Kaggle's four generic section labels to four real Dubai communities from the RTA data, selected to represent distinct parking contexts across the city:

Kaggle Section	Dubai Community	Context
Zone A	AL RAS	Historic Deira district — high-density street parking
Zone B	BURJ KHALIFA	Downtown Dubai — premium structured parking

Zone C	AL SATWA	Mixed residential and commercial neighbourhood
Zone D	HOR AL ANZ	Northern Dubai — large-capacity community zone

Engineered Features

Feature	Type	Logic	Purpose
is_occupied	SMALLINT (binary)	1 if occupancy_status = 'Occupied', else 0	Enables fast SUM/AVG aggregations in SQL
peak_hour_flag	SMALLINT (binary)	1 if hour is 7–9 or 17–19, else 0	Identifies peak demand periods for trend analysis

5. Orchestration & Quality Assurance

Pipeline Orchestration

The ETL pipeline is executed as a standalone Python script (etl_pipeline.py). Each stage runs sequentially and prints status output, allowing the operator to monitor progress and catch errors at each step. The pipeline is unaltered – it drops and recreates tables on each run, ensuring clean state for repeated executions.

Data Quality Checks

Three automated quality checks run after the transformation stage and before any data is loaded into PostgreSQL. If any check fails, the pipeline reports the issue and stops before writing to the database:

Check	Description	Expected Result
Null zone IDs	Verifies every fact row maps to a valid zone via the lookup table. Null zone_ids indicate a failed mapping.	0 null values
Duplicate sensor readings	Ensures no parking spot logs two different states at the same timestamp, which would indicate sensor error or data duplication.	0 duplicates
Invalid occupancy values	Validates that occupancy_status contains only 'Occupied' or 'Vacant' – rejects any unexpected values from source data.	0 invalid rows

Pipeline run results confirmed all three checks passed: 0 null zone IDs, 0 duplicate readings, and 0 invalid occupancy values across all 1,000 loaded rows.

Referential Integrity

The PostgreSQL schema enforces referential integrity at the database level: the `zone_id` column in `fact_occupancy` is defined as a foreign key referencing `dim_zones(zone_id)`. This prevents orphaned records and ensures every occupancy event can always be joined to its zone metadata.

6. Results & Analytical Insights

Pipeline Output

The pipeline successfully loaded 83 zone records into `dim_zones` and 1,000 occupancy events into `fact_occupancy`. The Streamlit dashboard confirms the following key metrics across all zones:

Metric	Value
Total sensor events loaded	1,000
Occupied events	550 (55.0%)
Vacant events	450 (45.0%)
Average occupancy rate	55.0%
Average parking duration	2.9 minutes
Peak hour events	382

Zone-Level Occupancy

Zone	Total Capacity	Events	Avg Occupancy %
AL RAS	1,441	382	57.1%
BURJ KHALIFA	365	307	56.7%
HOR AL ANZ	4,045	114	54.4%
AL SATWA	997	197	48.7%

Key Findings

- Al Ras records the highest average occupancy (57.1%), consistent with its role as a dense historic Deira neighborhood with limited structured parking supply
- Burj Khalifa zone follows closely at 56.7%, reflecting high visitor and commercial demand in Downtown Dubai
- Al Satwa shows the lowest occupancy at 48.7%, suggesting relatively adequate supply for its mixed residential-commercial land use

- Peak hour events (07:00–09:00, 17:00–19:00) account for 38.2% of all recorded events, confirming a clear morning and evening demand pattern
- Vehicle type distribution is heavily skewed, with one category representing approximately 70% of all events — suggesting a dominant vehicle class in the monitored zones

7. Challenges & Recommendations

Technical Challenges

Schema mismatch between datasets: The primary engineering challenge was the absence of a shared natural key between the RTA and Kaggle datasets. The RTA data uses Dubai community names while the Kaggle dataset uses abstract section labels. Resolution required designing a manual zone lookup table — a deliberate engineering decision rather than a limitation. Future iterations could automate this mapping using geographic coordinates and spatial joins if GPS data is available in the sensor dataset.

PostgreSQL authentication configuration: During environment setup, the initial PostgreSQL installation did not prompt for a superuser password, causing authentication failures. This was resolved by modifying the `pg_hba.conf` authentication file to temporarily allow trust-based connections, resetting the password via pgAdmin, and restoring `scram-sha-256` authentication. This highlighted the importance of environment documentation in reproducible pipeline deployments.

Synthetic dataset characteristics: The Kaggle IIoT dataset is synthetically generated rather than sourced from real Dubai sensors. While this does not affect the technical validity of the pipeline, it means the occupancy patterns and distributions may not reflect real-world Dubai parking behaviour. A production deployment would replace this with live RTA sensor feeds.

Recommendations

- Integrate a real-time RTA sensor API feed to replace the static Kaggle CSV, enabling live occupancy monitoring
- Add Apache Airflow scheduling to automate pipeline runs on a defined cadence (e.g., hourly ingestion of sensor updates)
- Expand the star schema with additional dimension tables: `Dim_Time` (hour, day, week, month hierarchy) and `Dim_Vehicle` (vehicle category metadata)
- Implement coordinate-based spatial mapping to automate zone matching using GPS data, replacing the manual lookup table
- Add a processed data export stage that writes cleaned CSVs to the `data/processed/` folder for audit and reproducibility

8. References & Supporting Material

References

Dubai Pulse. (2026). Number of parking spaces per zone [Dataset]. Roads and Transport Authority (RTA), Government of Dubai. <https://data.dubai.gov.ae>

DatasetEngineer. (2024). Smart Parking Management Dataset [Dataset]. Kaggle. <https://www.kaggle.com/datasets/datasetengineer/smart-parking-management-dataset>

Pandas Development Team. (2024). pandas: Powerful Python data analysis toolkit (Version 2.x) [Software]. <https://pandas.pydata.org>

PostgreSQL Global Development Group. (2024). PostgreSQL 17 documentation. <https://www.postgresql.org/docs/17/>

SQLAlchemy Authors. (2024). SQLAlchemy — The Python SQL toolkit and object relational mapper. <https://www.sqlalchemy.org>

Streamlit Inc. (2024). Streamlit documentation. <https://docs.streamlit.io>

Supporting Materials

- `etl_pipeline.py` — Full ETL pipeline source code
- `dashboard.py` — Streamlit analytics dashboard
- `main_pipeline.ipynb` — Jupyter notebook with annotated pipeline walkthrough
- `db_schema.sql` — PostgreSQL schema definition with indexes and analytical views
- `diagrams/erd_model.png` — Entity-relationship diagram
- `diagrams/architecture.png` — Pipeline architecture diagram
- `README.md` — Setup and deployment guide